

Machine Learning

FRANCESCO RUSSO¹

October 1, 2026

¹<https://github.com/francescorus>

This document consists of personal, independent, and unofficial lecture notes and summaries covering topics from the **Machine Learning** course for the **Computer Engineering** degree program. This material is created solely to support individual study and is shared free of charge for peer collaboration. Sale or commercial distribution is strictly prohibited.

Introduction

Note: The core course notes officially begin in Chapter 1 (the preceding sections serve as introductory background, if you want you can skip it!). Machine Learning provides frameworks and set of tools to tackle decision making tasks under limited information provided by measured data. Components of learning:

- 1) Input $x \in X$ [Vector of p values from which we want to predict]
- 2) Output $y \in Y$ [value we want to predict]

we would like to learn target function: $f : X \rightarrow Y$

Data = set of examples: $(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)$ is the TRAINING SET (e.g. i.i.d. sample from distribution D).

We can give after learning from training set an Hypothesis Function that is as similar as function f . $h : X \rightarrow Y$ [Model classes H , h in H]

There are many ways to learn: Classification (supervised and Y discrete), Regression (supervised and Y continuous), Clustering (Unsupervised and Y discrete), Dimensionality reduction (Unsupervised and Y continuous).

In order to deep dive in Machine Learning arguments we need to summarize three pillars that founds ML: Probability/Statistics, Algebra and Optimization, and Computation (algorithms and data structures).

Statistics

Estimating the mean of a Gaussian Random Variable

We observe a dataset of i.i.d (independent, identical, distributed) random variable, and we assume that $X_i \sim \mathcal{N}(\theta, \sigma^2)$. We want to estimate θ using measurements $x_1, \dots, x_m$. What we have to do is:

1. Find a point estimator ($\hat{\theta} = f(X_1, \dots, X_m)$)
2. Understand if the point estimator is close or far from the real mean θ
3. Decide if it's reasonable that θ is a given value (my analysis drive me to say that my random variable has e.g. $\theta = 0$)

The Probability answers the question "If I know the parameter θ , how likely am i to observe data x ?" (so the probability to see x). The Likelihood, instead, flips the question: "I already collected data $x = [x_1, \dots, x_m]$ but i don't know the true parameter θ . Which parameter value θ makes my observed data most plausible? So i need to find θ that makes the probability correct.

$p_X(x; \theta)$ (Likelihood Function) is the density probability (a number when we choose x and θ) of the vector x , parameterized by θ (unknown). It represents the probability (of the random variable X) of x (the values that we measured) changing as we want the mean (θ). Now that we know the density family of our random variable we define the **Log Likelihood Function** as:

$$L(\theta) = \log p_X(x; \theta)$$

we used logarithm because products of probability densities turn into sums.

Let's make an example: for X_i , i.i.d. $X_i \sim \mathcal{N}(\theta, \sigma^2)$ we have:

$$p_{X_i}(x_i; \theta) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x_i - \theta)^2}{2\sigma^2}\right)$$

$$L(\theta) := \log p_X(x; \theta) = \log\left(\prod_{i=1}^m p_{X_i}(x_i; \theta)\right) = -\frac{m}{2} \log(2\pi\sigma^2) - \frac{1}{2} \sum_{i=1}^m \frac{(x_i - \theta)^2}{\sigma^2}$$

(see passages as exercise)

The **Maximum Likelihood Estimator** of θ is defined as:

$$\hat{\theta} := \arg \max_{\theta} L(\theta)$$

In brief, $\hat{\theta}$ is the specific parameter value that maximizes the log-Likelihood function $L(\theta)$. Makes sense to search for maximum values of θ , because only the real mean maximises $L(\theta)$ with m large enough.

e.g

$$\hat{\theta} := \arg \max_{\theta} L(\theta) = \frac{1}{m} \sum_{i=1}^m x_i$$

which is nothing but the simple mean. Just for clear: if i take only one measure (one value of x , so an array with one value) the $\hat{\theta}$ that maximize the $p_x(x; \theta)$ is x !

Point Estimator itself is not enough, because itself is a random variable. If we repeat the experiment tomorrow with a new batch of measurements, we will get slightly different value for $\hat{\theta}$. A single point estimates don't tell us how close to θ is. We need to find, based on measured data, an interval or region $I(X)$ around $\hat{\theta}$ that is likely to enclose the true parameter θ . $I(X)$ is a random set, while θ is fixed. $I(X)$ is a **confidence interval** (for θ) of level $1 - \alpha$, if

$$\Pr[\theta \in I(X)] > 1 - \alpha$$

We could say that, out of 100 independent draws of $x \sim p_x(x; \theta)$, we expect that $(1 - \alpha)$ of the times (in 100 draws) the interval $I(X)_x = x$ contains the true (but unknown) θ . $I(X)$ is like an offset for $\hat{\theta}$ based on X ($I(X) = \text{Center} \pm \text{Offset}$). We would like the set $I(X)$ to be: small and contain θ with high probability (α small). With this features we are able to locate the parameter with high precision and high confidence.

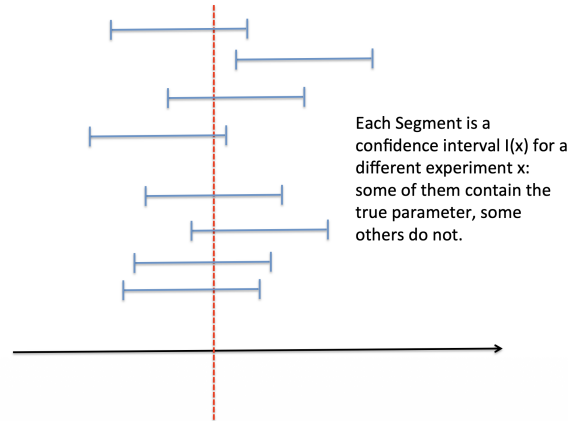


Figure 1: Example

So, as we said the point estimator is itself a random variable: $\hat{\theta} = \frac{1}{m} \sum_{i=1}^m x_i \sim \mathcal{N}(\theta, \frac{\sigma^2}{m})$. It's easy to show (HOMEWORK) that, for any given confidence $1-\alpha$, the smallest confidence set for θ is symmetric and centered around the sample mean:

$$I(X) = [\hat{\theta} - z_{\frac{\alpha}{2}} \frac{\sigma}{\sqrt{m}}, \hat{\theta} + z_{\frac{\alpha}{2}} \frac{\sigma}{\sqrt{m}}]$$

Sometimes we are interested in verifying/falsifying an assumption (the so called null hypothesis H_0) on the parameter $\hat{\theta}$. For example, we would like to test the hypothesis $\theta = \theta_0$ (θ_0 , fixed and known). Before collecting the data, we divide all possible experimental outcomes in two zones: Acceptance region (outcomes that are plausible and expected if H_0 is true), Rejection Region $R(H_0)$ Outcomes that are so extreme or unexpected if H_0 were true then makes we say "I don't believe H_0 anymore". To do that we introduce the **rejection region**, a function of the null hypothesis. given a sample $x \sim p_x(x; \theta)$, if: $x \in R(H_0) \implies \text{reject } H_0$.

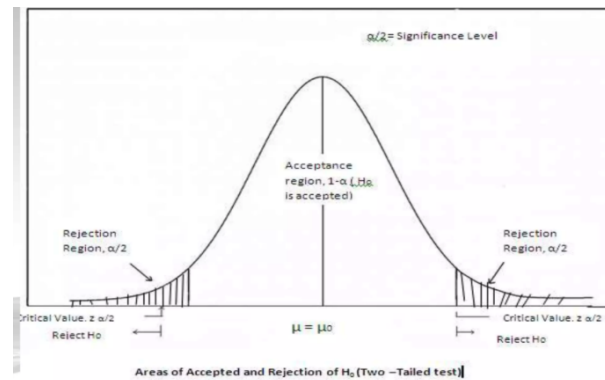


Figure 2: In case my distribution has μ_0 as mean i don't accept x values in the rejection region (it's the same also in the negative part of the function), this because too far from the mean

This function can lead to two types of error:

1. $x \in R(H_0)$ when H_0 is *true*
2. $x \notin R(H_0)$ when H_0 is *false*

The probabilities of these two errors are typically denoted as

$$P[x \in R(H_0) | H_0 = \text{true}] = \alpha$$

and

$$P[x \notin R(H_0) | H_0 = \text{false}] = \beta$$

Notice that the first sentence is easier than the second to calculate. Another thing to say is: if i fix α , there are still infinity rejection region. this because i impose the "length" of the rejection region, not how it is done. For example if i have $P[x \in R(H_0) | H_0 = \text{true}] = 0.05$ This mean that under the null hypothesis distribution the total area under the probability density curve over the region $R(H_0)$ must equal 5 percent. You could pick the 5 percent tail on the far right, or far left, or you could split 2.5 percent on the left and 2.5 percent on the right.

Let's go back to our example: we consider Example 1 above and for simplicity assume $\sigma^2 = 1$). We would like to test the hypothesis $H_0 : \theta = \theta_0 = 0$ (that the mean of X_i is zero). The theory of statistical testing (for those of you interested look for Neyman-Pearson theorem) guarantees that the *biggest* region $\mathcal{R}(H_0)$ for a given *type-I* error probability α is of the form

$$\mathcal{R}(H_0) = \left\{ (x_1, \dots, x_m) : |\hat{\theta}| := \left| \frac{1}{m} \sum_{i=1}^m (x_i - \theta_0) \right| > z_{\alpha/2} \frac{1}{\sqrt{m}} \right\}$$

If we assume $\theta_0 = 0$

$$\mathcal{R}(H_0) = \left\{ (x_1, \dots, x_m) : |\hat{\theta}| := \left| \frac{1}{m} \sum_{i=1}^m x_i \right| > z_{\alpha/2} \frac{1}{\sqrt{m}} \right\}$$

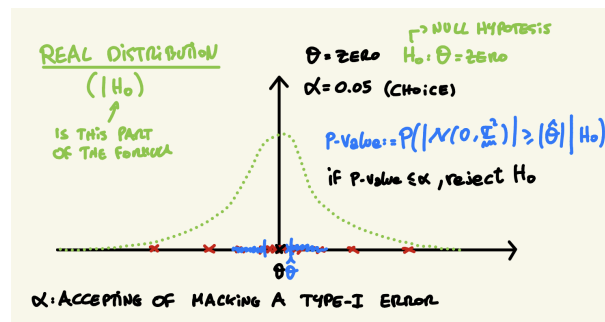
Since the sample estimator satisfies $\hat{\theta} \sim \mathcal{N}\left(\theta, \frac{1}{m}\right)$, it is straightforward to verify that under the null hypothesis $H_0 : \theta = 0$, the probability of committing a Type I error matches the nominal significance level α :

$$\mathbb{P}\left[|\hat{\theta}| > \frac{z_{\alpha/2}}{\sqrt{m}} \mid H_0\right] = \alpha.$$

Equivalently, rather than comparing the test statistic directly to a predetermined critical value, the hypothesis test can be formulated in terms of the **p-value**. Given the observed sample realization $\bar{x} = \frac{1}{m} \sum_{i=1}^m x_i$, the *p-value* represents the probability of observing a test statistic at least as extreme as the one actually computed from the data, assuming H_0 is true:

$$p := \mathbb{P}\left[|Z| > \left| \frac{1}{m} \sum_{i=1}^m x_i \right| \mid H_0\right], \quad \text{where } Z \sim \mathcal{N}\left(0, \frac{1}{m}\right).$$

The p-value is the probability of observing a sample mean at least as extreme as your actual measured value $\hat{\theta}$, assuming that H_0 is true.



What is the relationship between confidence intervals and hypothesis testing? There is a direct duality between the two concepts: Testing H_0 at significance level α is equivalent to checking whether θ_0 falls inside $(1 - \alpha)$ confidence interval $I(X)$.

Linear Regression with Known Variance σ^2

to be continued...

Chapter 1

In supervised learning, our goal is to infer a rule from observed data. We start by formally defining the spaces:

- **Domain Set ($\mathcal{X}$):** The space of all possible inputs (features).
- **Label Set ($\mathcal{Y}$):** The space of all possible outputs (targets). Depending on the problem, $\mathcal{Y}$ can be:
 - *Discrete*: in classification problems (e.g., recognizing handwritten digits from an image).
 - *Continuous*: in regression problems.

The learner does not see the entire world, but only a finite set of labeled examples called the **Training Set** (or sample set):

$$S := \{(x_i, y_i)\}_{i=1}^m$$

where m denotes the sample size.

How does the environment generate data?

To analyze learning mathematically, we must make assumptions about how the training set S is produced:

1. **Simple Assumption (Deterministic Labels):** Each instance x_i is sampled independently and identically distributed (i.i.d.) from a probability distribution $\mathcal{D}_x$, and there exists an unknown deterministic ground-truth function $f(x)$ that maps each x_i to its exact label:

$$x_i \sim \mathcal{D}_x \quad \text{and} \quad \exists f(x) \text{ such that } y_i = f(x_i)$$

2. **Realistic Assumption (Probabilistic Labels):** In the real world, identical inputs may have different labels due to noise or missing information. Thus, the output y is also governed by a distribution conditioned on x :

$$y \mid x \sim \mathcal{D}_{y|x}$$

Key Insight: The *Realistic* setting is the general framework. The *Simple* setting is just a special case where $\mathcal{D}_{y|x}$ is completely deterministic—meaning all the probability mass is concentrated at the single value $f(x)$ (conceptually like a Dirac delta δ function).

2. The Learning Algorithm and Loss Functions

Under the deterministic assumption, our concrete goal is: **learn $f(x)$ from empirical measurements to predict y for unseen inputs x .**

A **Learning Algorithm** A takes the training set S as input and outputs a hypothesis function h from a hypothesis class $\mathcal{H}$:

$$A(S) \rightarrow h \in \mathcal{H}$$

Here, h acts as our best candidate approximation of the true function f .

Quantifying Error: The Loss Function

To measure how well a chosen hypothesis h predicts y , we define a **Loss Function** $\ell(h, x)$.

Important distinction: The loss function evaluates the discrepancy on a **single sample**!

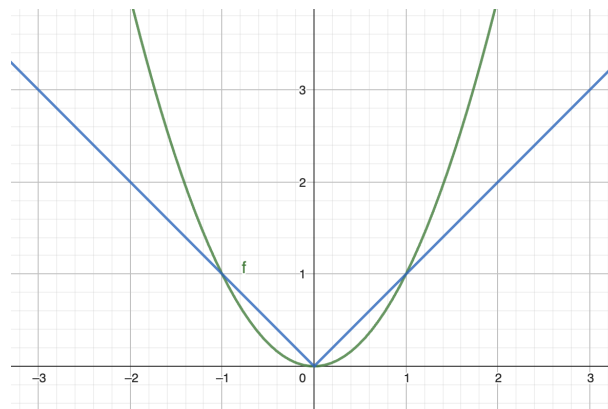
Example 1: Classification

The most standard choice in classification is the **0–1 loss function**:

$$y \in \{0, 1\}, \quad \ell_{0-1}(h, x) = \begin{cases} 0 & \text{if } h(x) = y \quad (\text{correct classification}) \\ 1 & \text{if } h(x) \neq y \quad (\text{misclassification}) \end{cases} \quad (1)$$

Example 2: Quadratic Loss vs. Absolute Loss

The mathematical choice of the loss function heavily affects how the algorithm treats mistakes:



Consider comparing:

$$\text{Squared Error: } \ell(h, x) = (y - h(x))^2 \quad (y \in \{0, 1\} \text{ or } \mathbb{R})$$

vs.

$$\text{Absolute Error (Modulus): } \ell(h, x) = |y - h(x)| \quad (y \in \mathbb{R})$$

Takeaway for students:

- The **quadratic loss** penalizes larger errors much more heavily (growing quadratically), making it sensitive to outliers.
- The **modulus loss** increases linearly with the error magnitude, maintaining constant sensitivity regardless of scale.

3. From Single Samples to Global Error: True Risk vs. Empirical Risk

Evaluating the error on a single instance (x, y) is not enough: we want to know how well h performs on average across all possible instances in the environment. This average is the **True Risk** (or Generalization Error):

$$L_{\mathcal{D}}(h) := \mathbb{E}_{(x,y) \sim \mathcal{D}}[\ell(h, x)]$$

$L_{\mathcal{D}}(h)$ represents the true, expected error of h over the entire distribution $\mathcal{D}$. Therefore, the ideal learning algorithm would pick the hypothesis that minimizes this quantity:

$$A(S) = \arg \min_{h \in \mathcal{H}} L_{\mathcal{D}}(h)$$

The Big Problem

To compute $L_{\mathcal{D}}(h) = \mathbb{E}_{(x,y) \sim \mathcal{D}}[\ell(h, (x, y))]$, we would need complete knowledge of the underlying distribution $\mathcal{D}$. But in real-world machine learning, $\mathcal{D}$ is **unknown**!

The Solution: Empirical Risk

Since we cannot compute the theoretical expectation directly, we compute the empirical sample mean over our available training data S .

By the **Law of Large Numbers (LLN)**, as the sample size grows to infinity ($m \rightarrow +\infty$), the sample mean $\hat{\theta}$ converges to the true population mean θ :

$$\hat{\theta} \approx \theta$$

This justifies calculating the **Empirical Risk** $L_S(h)$ directly from our dataset:

$$L_S(h) := \frac{1}{m} \sum_{i=1}^m \ell(h, (x_i, y_i))$$

—

4. The Ultimate Goal: PAC Learning

Our ultimate goal is **not** just to minimize $L_S(h)$ on the training data (doing so blindly can lead to *overfitting*, which we will analyze later), but to guarantee that the **True Risk** $L_{\mathcal{D}}(h)$ is as **small as possible**.

Because the training set S is drawn randomly, there is always a tiny probability of drawing an unrepresentative sample. Therefore, we look for guarantees of the form:

$$\mathbb{P}[\text{True Risk } L_{\mathcal{D}}(A(S)) \text{ is small}] \text{ is large}$$

In mathematical terms, we want:

$$\mathbb{P}[L_{\mathcal{D}}(A(S)) \leq \epsilon] \geq 1 - \delta$$

This core framework is known as **PAC Learning** (*Probably Approximately Correct Learning*):

- **Approximately Correct:** The error $L_{\mathcal{D}}(h)$ is bounded by a small threshold ϵ .
- **Probably:** This condition holds with high confidence $1 - \delta$.

To be continued